

EXERCÍCIO PRÁTICO 01 – COLETOR OPENF1 → MONGODB

Implementação e Gerenciamento de Bancos de Dados NoSQL

Código no GitHub: github.com/joseniltoncirne/Implementacao-e-Geren-de-banco-de-dados/tree/main/individuais/exercicio01-openf1

O que o script faz

O script `f1_data_collector.py` pega os dados do GP da Itália de 2023 (Monza) no site da OpenF1 e guarda tudo no MongoDB, no banco `openf1_data`. Os dados ficam separados em 3 coleções:

- **sessions:** as sessões do fim de semana (treinos, classificação e corrida).
- **drivers:** os pilotos que correram.
- **laps:** as voltas de cada piloto na corrida.

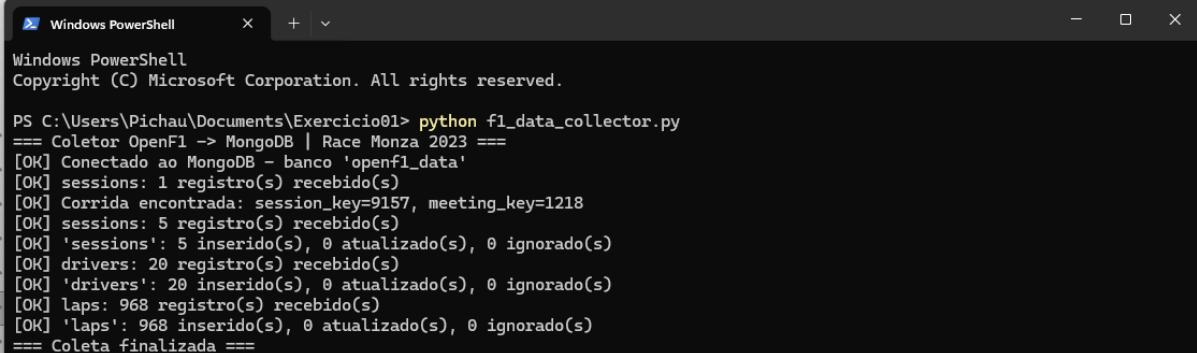
Um detalhe importante: o enunciado cita os números `session_key=9159` e `meeting_key=1219`, mas conferindo na API esses números são do GP de Singapura, e não do GP da Itália. Por isso o script procura a corrida pelo nome: ano 2023, circuito de Monza, sessão Race. Assim ele acha a corrida certa sozinho (`session_key=9157`).

Para não repetir dados, antes de salvar o script confere se aquele registro já está no banco. Se já estiver, ele só atualiza. Se não estiver, ele cria. Por isso dá para rodar o script quantas vezes quiser sem ficar nada duplicado.

1. Instalando o que precisa e rodando pela primeira vez

```
python -m pip install -r requirements.txt
python f1_data_collector.py
```

Primeiro o script acha a corrida de Monza. Na primeira vez, tudo é novo e entra no banco: 5 sessões, 20 pilotos e 968 voltas.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Pichau\Documents\Exercicio01> python f1_data_collector.py
=== Coletor OpenF1 -> MongoDB | Race Monza 2023 ===
[OK] Conectado ao MongoDB - banco 'openf1_data'
[OK] sessions: 1 registro(s) recebido(s)
[OK] Corrida encontrada: session_key=9157, meeting_key=1218
[OK] sessions: 5 registro(s) recebido(s)
[OK] 'sessions': 5 inserido(s), 0 atualizado(s), 0 ignorado(s)
[OK] drivers: 20 registro(s) recebido(s)
[OK] 'drivers': 20 inserido(s), 0 atualizado(s), 0 ignorado(s)
[OK] laps: 968 registro(s) recebido(s)
[OK] 'laps': 968 inserido(s), 0 atualizado(s), 0 ignorado(s)
=== Coleta finalizada ===
```

2. Rodando de novo para ver se duplica

```
python f1_data_collector.py
```

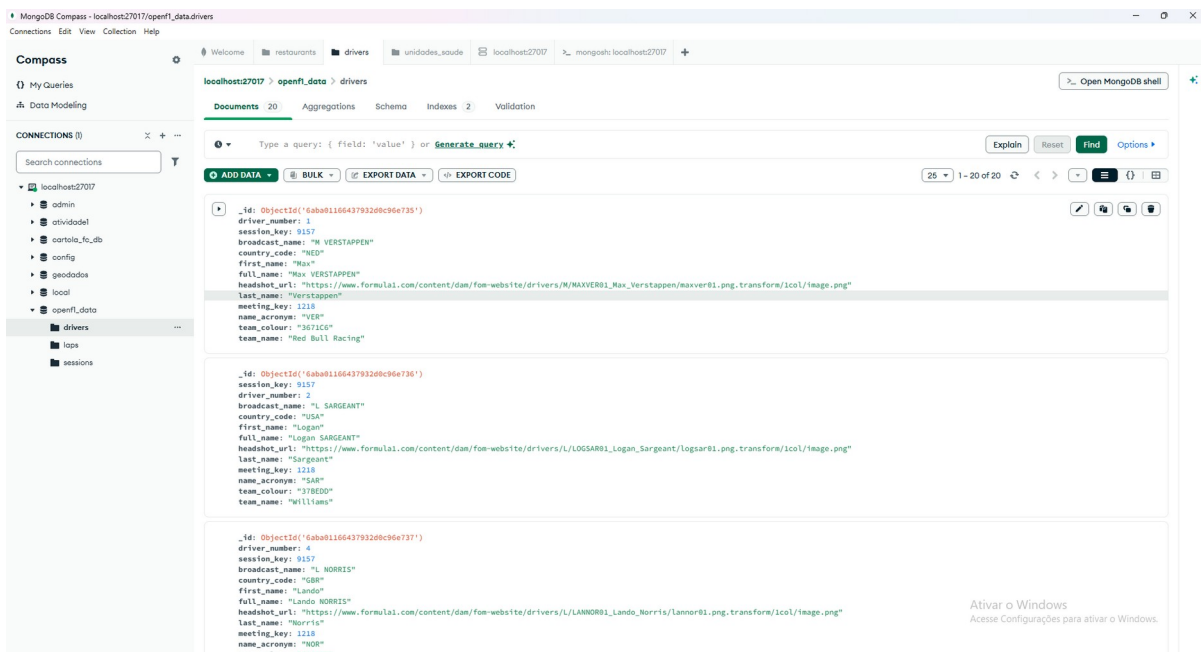
Na segunda vez, nada novo entra (0 inserido) e tudo só é atualizado. Ou seja: rodar de novo não cria nada repetido.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Pichau\Documents\Exercicio01> python f1_data_collector.py
=== Coletor OpenF1 -> MongoDB | Race Monza 2023 ===
[OK] Conectado ao MongoDB - banco 'openf1_data'
[OK] sessions: 1 registro(s) recebido(s)
[OK] Corrida encontrada: session_key=9157, meeting_key=1218
[OK] sessions: 5 registro(s) recebido(s)
[OK] 'sessions': 5 inserido(s), 0 atualizado(s), 0 ignorado(s)
[OK] drivers: 20 registro(s) recebido(s)
[OK] 'drivers': 20 inserido(s), 0 atualizado(s), 0 ignorado(s)
[OK] laps: 968 registro(s) recebido(s)
[OK] 'laps': 968 inserido(s), 0 atualizado(s), 0 ignorado(s)
=== Coleta finalizada ===
PS C:\Users\Pichau\Documents\Exercicio01> python f1_data_collector.py
=== Coletor OpenF1 -> MongoDB | Race Monza 2023 ===
[OK] Conectado ao MongoDB - banco 'openf1_data'
[OK] sessions: 1 registro(s) recebido(s)
[OK] Corrida encontrada: session_key=9157, meeting_key=1218
[OK] sessions: 5 registro(s) recebido(s)
[OK] 'sessions': 0 inserido(s), 5 atualizado(s), 0 ignorado(s)
[OK] drivers: 20 registro(s) recebido(s)
[OK] 'drivers': 0 inserido(s), 20 atualizado(s), 0 ignorado(s)
[OK] laps: 968 registro(s) recebido(s)
[OK] 'laps': 0 inserido(s), 968 atualizado(s), 0 ignorado(s)
=== Coleta finalizada ===
PS C:\Users\Pichau\Documents\Exercicio01> |
```

3. Dados salvos no MongoDB

Abrindo o MongoDB Compass, dá para ver o banco openf1_data com as 3 coleções e os 20 pilotos salvos:



Código

f1_data_collector.py

```
import os
import sys
import time

import requests
from dotenv import load_dotenv
from pymongo import MongoClient, ASCENDING
from pymongo.errors import PyMongoError

load_dotenv()

API_BASE_URL = os.getenv("OPENF1_BASE_URL", "https://api.openf1.org/v1")
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
DB_NAME = os.getenv("MONGO_DB_NAME", "openf1_data")
YEAR = int(os.getenv("YEAR", "2023"))
CIRCUIT = os.getenv("CIRCUIT", "Monza")
SESSION_NAME = os.getenv("SESSION_NAME", "Race")
REQUEST_TIMEOUT = int(os.getenv("REQUEST_TIMEOUT", "30"))
MAX_RETRIES = int(os.getenv("MAX_RETRIES", "3"))

DATASETS = {
    "sessions": {"collection": "sessions", "unique_keys": ["session_key"]},
    "drivers": {"collection": "drivers", "unique_keys": ["session_key", "driver_number"]},
    "laps": {"collection": "laps", "unique_keys": ["session_key", "driver_number",
"lap_number"]},
}

def get_database(uri: str = MONGO_URI, db_name: str = DB_NAME):
    try:
        client = MongoClient(uri, serverSelectionTimeoutMS=5000)
        client.admin.command("ping")
        print(f"[OK] Conectado ao MongoDB - banco '{db_name}'")
        return client[db_name]
    except PyMongoError as e:
        print(f"[ERRO] Não foi possível conectar ao MongoDB: {e}")
        sys.exit(1)

def fetch_data(endpoint: str, params: dict) -> list:
    url = f"{API_BASE_URL}/{endpoint}"

    for attempt in range(1, MAX_RETRIES + 1):
        try:
            response = requests.get(url, params=params, timeout=REQUEST_TIMEOUT)

            if response.status_code == 429 or response.status_code >= 500:
                wait = 2 ** attempt
                print(f"[AVISO] {endpoint}: HTTP {response.status_code}, "
                    f"tentativa {attempt}/{MAX_RETRIES}, aguardando {wait}s...")
                time.sleep(wait)
                continue

            response.raise_for_status()
            data = response.json()

            if isinstance(data, dict):
                data = [data]

            print(f"[OK] {endpoint}: {len(data)} registro(s) recebido(s)")
            return data

        except requests.exceptions.Timeout:
            print(f"[AVISO] {endpoint}: timeout (tentativa {attempt}/{MAX_RETRIES})")
            time.sleep(2 ** attempt)
        except requests.exceptions.RequestException as e:
            print(f"[ERRO] Falha na requisição a {url}: {e}")
            return []
        except ValueError:
            print(f"[ERRO] Resposta de {url} não é um JSON válido")
            return []
```

```

    print(f"[ERRO] {endpoint}: desistindo após {MAX_RETRIES} tentativas")
    return []

def ensure_unique_index(collection, unique_keys: list) -> None:
    collection.create_index([(key, ASCENDING) for key in unique_keys], unique=True)

def save_to_collection(db, data: list, collection_name: str, unique_keys: list) -> None:
    if not data:
        print(f"[AVISO] Nada para salvar em '{collection_name}'")
        return

    collection = db[collection_name]
    inserted = updated = skipped = 0

    try:
        ensure_unique_index(collection, unique_keys)

        for doc in data:
            if any(doc.get(key) is None for key in unique_keys):
                skipped += 1
                continue

            filtro = {key: doc[key] for key in unique_keys}
            result = collection.update_one(filtro, {"$set": doc}, upsert=True)

            if result.upserted_id is not None:
                inserted += 1
            else:
                updated += 1

        print(f"[OK] '{collection_name}': {inserted} inserido(s), "
              f"{updated} atualizado(s), {skipped} ignorado(s)")

    except PyMongoError as e:
        print(f"[ERRO] Falha ao salvar em '{collection_name}': {e}")

def _cfg(endpoint: str) -> dict:
    return {"collection_name": DATASETS[endpoint]["collection"],
            "unique_keys": DATASETS[endpoint]["unique_keys"]}

def main() -> None:
    print(f"=== Coletor OpenF1 -> MongoDB | {SESSION_NAME} {CIRCUIT} {YEAR} ===")
    db = get_database()

    corrida = fetch_data("sessions", {"year": YEAR, "circuit_short_name": CIRCUIT,
    "session_name": SESSION_NAME})
    if not corrida:
        print("[ERRO] Corrida não encontrada na API. Confira YEAR, CIRCUIT e SESSION_NAME.")
        return
    session_key = corrida[0]["session_key"]
    meeting_key = corrida[0]["meeting_key"]
    print(f"[OK] Corrida encontrada: session_key={session_key}, meeting_key={meeting_key}")

    sessions = fetch_data("sessions", {"meeting_key": meeting_key})
    save_to_collection(db, sessions, *_cfg("sessions"))

    drivers = fetch_data("drivers", {"session_key": session_key})
    save_to_collection(db, drivers, *_cfg("drivers"))

    laps = fetch_data("laps", {"session_key": session_key})
    save_to_collection(db, laps, *_cfg("laps"))

    print("=== Coleta finalizada ===")

if __name__ == "__main__":
    main()

```

requirements.txt

```
requests  
pymongo  
python-dotenv
```